

NullShadow v4.0 — Implementation Brief

Handoff document for continuing development in a new Claude conversation

Context for Claude: This is a continuation brief for an existing project called NullShadow, a Python-based CLI penetration testing framework (FYP by Tayyab, BS Cybersecurity). The repository already exists on GitHub at github.com/rajatayyabb/NullShadow-v3-DL-RL with a working v3.0 codebase (33 tools across pentesting, OSINT, AI/utilities, and DL/RL demo modules). This brief describes the planned v4.0 additions, in order, with the reasoning behind each so you can implement them consistently with the existing codebase rather than rewriting it from scratch.

Read the attached repository files first (see Section 5: Required Attachments) before writing any code, so new modules match existing naming conventions, the database schema in `null_db/db.py`, and the AI engine interfaces in `modules/ai/`.

API & Cost Safety — Read Before Testing Any Code

- Cloud AI APIs (Claude, GPT-4, Gemini via `modules/ai/ai_engine.py`) require paid keys. Do NOT call `ai_engine.py` functions during development/testing unless the user confirms a funded key is configured. Default to `modules/ai/local_ai_engine.py` (Ollama) or stub/mock the AI call when testing new modules in Phases 1-4.
- The NVD CVE API (Phase 1) is free and keyless, but rate-limited to 5 requests per 30 seconds without a key. Any function that loops over multiple CVE IDs MUST include caching (check the local `cve_cache` table before calling the API) and a delay/backoff between requests. Do not write a loop that calls `lookup_cve()` for every finding in a large scan without this; it will return 403/429 errors and may appear as a 'broken' feature when it is actually a rate limit.
- If any test run produces an authentication, billing, or 429/403 error, treat it as an expected API-limit issue first (check key configuration and rate limits) before assuming the surrounding logic is broken.

1. Current State (v3.0) — What Already Exists

The existing codebase has the following structure. Do not restructure this unless necessary; new code should integrate into it.

Path	Purpose
main.py	CLI entry point and menu (33 numbered tools)
config/config.py	API keys, DB_PATH, LOCAL_AI_URL and other settings
null_db/db.py	SQLite wrapper (ScanDatabase class) — stores scans table with target, scan_type, timestamp, results (JSON), ai_analysis, report_path, threat_score
modules/ai/ai_engine.py	Cloud AI engine (Claude/GPT-4/Gemini)
modules/ai/local_ai_engine.py	Local AI via Ollama
modules/ai/deep_learning_engine.py	Simulated GNN-based vulnerability scoring
modules/ai/orchestrator.py	RL-ready autonomous orchestrator
modules/ai/rl_engine.py	RL simulation logic
modules/ai/deception_engine.py	Honeypot/decoy demo
modules/ai/deanonymizer.py	De-anonymization helper module
modules/pentesting/scanner.py	Core scanning tools (port scan, vuln scan, etc.)
modules/pentesting/new_tools.py	Additional pentest/OSINT tools
modules/pentesting/iot_sec.py	IoT/firmware DL analysis + neural fuzzing
modules/osint/osint_tools.py	OSINT modules (WHOIS, geolocation, etc.)
modules/recon/recon_pipeline.py	Auto Full Recon pipeline
modules/reporting/report_generator.py	PDF report generation (ReportLab)
modules/utilities/util_tools.py	Misc utilities (password gen, network info, etc.)

Known issue: the current scans table stores results as a single JSON blob per scan with no structured per-finding records, no CVE cross-referencing, and no concept of comparing two scans of the same target over time. Section 2 (Phase 1) addresses this.

1.5 Phase 0: Critical Bug Fixes (Do This First, Before Any New Feature)

These are confirmed issues observed while running the existing v3.0 build on Kali Linux 2025.x. Fix these before starting Phase 1, and verify each fix by actually running the affected menu option afterward.

0.1 pip install fails with 'externally-managed-environment'

Kali 2025.x (Python 3.13) blocks system-wide pip installs per PEP 668. The setup.sh script's 'pip3 install -r requirements.txt' step fails silently and continues, meaning dependencies may not actually be installed even though setup.sh reports 'NullShadow v3.0 is ready'.

- Update setup.sh (or README) to create and use a virtual environment: `python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt`
- After this fix, verify by running 'python3 main.py' from within the venv and confirming options that depend on external packages (e.g. WHOIS, NVD lookups added in Phase 1) do not throw `ModuleNotFoundError`

0.2 WHOIS Lookup fails for non-generic TLDs (e.g. .edu.pk)

modules/recon/recon_pipeline.py `_whois()` and modules/osint/osint_tools.py `domain_whois()` both use a hardcoded RDAP server dict covering only com/net/org/io, falling back to `rdap.org` which does not reliably resolve country-code/educational TLDs like .edu.pk. Result: 'WHOIS unavailable' for many real domains.

- Add an IANA RDAP bootstrap lookup as the first method: fetch `https://data.iana.org/rdap/dns.json` (cache this file locally, it changes rarely), find the correct RDAP base URL for the target's TLD, then query `{base_url}/{domain}/{domain}`
- Keep the existing hardcoded dict and `rdap.org` fallback as secondary methods for when the bootstrap file can't be fetched (offline use)
- Apply this fix in both `recon_pipeline.py` and `osint_tools.py` (consider extracting a shared helper, e.g. `modules/intel/rdap_lookup.py`, used by both)
- Test with at least: a .com domain, a .edu.pk domain, and an unregistered/invalid domain (should return a clean 'not found' result, not a crash)

0.3 AI features default to cloud APIs and error without keys

Confirmed: AI Engine defaults to OPENAI on startup (shown in banner). If no `OPENAI_API_KEY` is set, any menu option that calls `modules/ai/ai_engine.py` directly (without checking key availability first) will throw an auth error instead of falling back.

- In `main.py` (or a shared helper), implement `get_active_ai_engine()`: if a cloud key is configured AND the user selected a cloud engine via [29] Switch AI Engine, return `ai_engine.py`'s engine; otherwise return `local_ai_engine.NullShadowAIEngine()`
- Update every AI-calling menu option ([25] AI Chat, [26] Auto Full Recon's analysis step, [30] Auto Pentest) to use `get_active_ai_engine()` instead of importing `ai_engine.py` directly
- Default banner AI Engine indicator should show LOCAL when no cloud key is configured, not OPENAI
- Test: with zero env vars / empty `config.py`, run [25], [26], and [30] and confirm none throw an authentication exception

0.4 Local AI gives only generic canned responses (Ollama not connected)

modules/ai/local_ai_engine.py's `_is_server_available()` check against `LOCAL_AI_URL/api/health` is likely returning False because Ollama is not installed/running, so every AI Chat response falls through to the generic `_fallback_response()` text regardless of the question asked.

- This is primarily an environment setup issue, not a code bug: document in README that Ollama must be installed and running (ollama serve) with a model pulled (e.g. `ollama pull llama3.2:3b`) for real AI responses
- Code improvement: NullShadowAIEngine's health check endpoint is `/api/health` but Ollama's actual endpoint is `/api/tags` or just checking `/` — verify against current Ollama API and fix `_is_server_available()` if the health check itself is incorrect (this may be the actual root cause, not just 'Ollama isn't running')
- Test: with Ollama running and a model pulled, confirm AI Chat responses vary based on the question (not the same canned text every time)

1.6 New Module: Guided Remediation (replaces/extends 'Explain Mode')

For every finding (from scanner.py, recon_pipeline.py, etc.), generate a structured remediation block via the AI engine (local or cloud, using the Phase 0.3 fallback chain). This is the 'best in class' differentiator: findings + explanation + general fix guidance, all in one place.

Scope — what this module DOES produce per finding:

- Plain-language explanation of the vulnerability/issue
- Relevant CWE and/or OWASP category reference
- Severity context (using CVSS score from Phase 1's NVD lookup if a CVE is present)
- General remediation steps (e.g. 'disable TLS 1.0/1.1, enable TLS 1.3, use modern cipher suites' for a weak-cipher finding)
- A generic configuration/code snippet showing the secure pattern (e.g. an nginx ssl_protocols line, or a parameterized-query example) — generic best-practice, not tailored to the specific scanned host

Scope — what this module explicitly DOES NOT do:

- Does NOT generate a host-specific patch/config diff for the actual scanned target
- Does NOT generate exploit code, working payloads, or step-by-step attack instructions for the discovered vulnerability
- Does NOT attempt to connect to or modify the scanned target in any way

Implementation notes:

- Create modules/intel/remediation.py with generate_remediation(finding_dict) that builds a constrained prompt and calls get_active_ai_engine() (from Phase 0.3)
- System prompt MUST explicitly instruct the model: explain the issue, map to CWE/OWASP, give general remediation steps and a generic secure-pattern snippet; do NOT produce exploit code, attack payloads, or target-specific instructions for the scanned host
- Integrate into modules/reporting/report_generator.py so each finding in the PDF report includes its remediation block
- Also usable standalone: add a menu option to run remediation generation on a single pasted finding/CVE for quick lookups

2. Planned v4.0 Work — Phases 1-6

Phase 0 (above) must be completed and verified first. The phases below proceed in order; each builds on the previous one. Do not skip ahead to MCP (Phase 4) before Phase 1 is working, since the MCP tools depend on the findings schema. Phase 6 (defensive monitoring dashboard) depends on Phase 1 (findings/CVSS data) and Phase 2 (scan diffing) and is the project's primary differentiator for academic presentation — if time is limited, Phases 1, 2, and 6 are the highest-priority combination, with Phases 3-5 as supporting/secondary work.

Phase 1: Findings Schema + NVD CVE Enrichment

Goal: normalize all module outputs into a common structured format, and enrich any CVE references with live data from the NVD API.

- Add a new table 'findings' to null_db/db.py (alongside the existing 'scans' table, do not remove it) with columns: id, scan_id (FK to scans.id), target, finding_type, severity, title, description, cve_ids (JSON list), raw_data (JSON), created_at
- Add helper methods to ScanDatabase: add_finding(), get_findings(target=None, scan_id=None), get_findings_by_cve(cve_id)
- Create modules/intel/nvd_lookup.py: a function lookup_cve(cve_id) that queries the NVD REST API (<https://services.nvd.nist.gov/rest/json/cves/2.0>) and returns severity (CVSS score), description, and published date. No API key required for low-volume use; add a note about NVD's rate limits (5 requests / 30s without a key) and basic caching (store results in a local 'cve_cache' table to avoid repeat lookups)
- Update existing scan modules (start with modules/pentesting/scanner.py vulnerability scanner and modules/osint/osint_tools.py CVE search) to call add_finding() for each result, and to call lookup_cve() when a CVE ID is present

Phase 2: Scan Diff Module

Goal: compare two scans of the same target and report what changed. Depends on Phase 1's findings table.

- Create modules/intel/scan_diff.py with a function diff_scans(target, scan_id_a, scan_id_b) that returns: new findings present in B but not A, resolved findings present in A but not B, and changed findings (same finding_type/title but different severity or details)
- Add a new menu option in main.py (e.g. [34] Scan Diff) that lists previous scans for a target (using get_history from db.py) and lets the user pick two to compare
- Output should be readable in the terminal (use rich Table, consistent with existing null_db/db.py display style) and optionally included in PDF reports via modules/reporting/report_generator.py

Phase 3: Exploit / PoC Reference Mapping

Goal: for each CVE found in a scan, look up whether a known Metasploit module or public exploit reference exists. This is informational only — reference lookup, not exploit code generation.

- Create modules/intel/exploit_refs.py with a function find_exploit_refs(cve_id) that checks a local reference table (modules/intel/data/msf_cve_map.json, a small curated mapping of CVE IDs to Metasploit module paths for common/well-known CVEs) and returns matches if found
- If no local match, the function should return a clear 'no known reference' result rather than attempting to search exploit databases live (keeps this offline-safe and avoids scope creep into exploit code retrieval)

- Integrate into the findings display: any finding with a matched CVE shows 'Known exploit reference: ' as informational context

Phase 4: MCP Server Layer

Goal: expose a focused subset of NullShadow's modules as MCP (Model Context Protocol) tools so MCP-compatible AI clients (Claude Desktop, Odysseus, etc.) can call them directly.

- Create `mcp_server.py` at the project root using the FastMCP Python framework (pip install `mcp` / `fastmcp`)
- Expose these existing functions as MCP tools (wrap, do not duplicate logic): `port_scan`, `vulnerability_scan`, `subdomain_enum`, `dns_recon`, `whois_lookup`, `hash_identify_crack`, `jwt_analyze`
- Each tool should return structured JSON (using the Phase 1 findings format) so the calling AI agent receives consistent data
- Add a `tools/claude_desktop_config.json` example showing how to register this server with Claude Desktop
- Keep the existing `main.py` CLI fully functional as a standalone mode; the MCP server is an additional interface, not a replacement

Phase 5: Local AI Only (Cost Control)

Goal: ensure the tool is fully usable with zero API costs, using the existing `local_ai_engine.py` and Ollama integration.

- `modules/ai/local_ai_engine.py` (NullShadowAIEngine) already includes a working fallback chain: it tries Ollama at `LOCAL_AI_URL`, and if unavailable, returns a rule-based canned response via `_fallback_response()`. This means local mode should never hard-error even with no Ollama running.
- Check every menu option in `main.py` that currently calls `modules/ai/ai_engine.py` (cloud Claude/GPT-4/Gemini) directly — especially [25] AI Chat Mode, [26] Auto Full Recon's AI analysis step, and [30] Auto Pentest (DL/RL). For each one, add a selection step: if a cloud API key is configured in `config.py` AND the user has selected a cloud engine via [29] Switch AI Engine, use `ai_engine.py`; otherwise, use `local_ai_engine.py`.
- The end state: with zero API keys configured, every AI-touching menu option still produces output (via `local_ai_engine.py`'s Ollama-or-fallback chain) instead of an authentication error.
- Update `config/config.example.py` so `LOCAL_AI_URL` is the default, and cloud API keys are clearly marked optional with a comment explaining the fallback behavior.
- Document in README: recommended local models (e.g. llama3.1:8b, qwen2.5:7b) and approximate hardware requirements, plus a note that the tool works with zero configuration via the rule-based fallback (lower quality, but functional).

Phase 6: Risk-Weighted Change Monitoring + Dashboard (Defensive Focus)

Goal: pivot NullShadow from a one-shot offensive scanner toward a defensive 'attack surface monitoring' tool — the kind of capability used in real blue-team/SOC contexts. Depends on Phase 1 (findings + CVSS data) and Phase 2 (scan diffing). This phase is intended to be the project's main differentiator for academic presentation: most student pentest tools are one-shot and offensive-only; this makes NullShadow continuous and defensive.

6.1 Scheduler

- Add a 'monitor' mode to main.py (e.g. `python3 main.py --monitor <target>`) that runs the existing Auto Full Recon pipeline (Phase 0.1-style, reusing `recon_pipeline.py`) and stores results via `add_finding()`
- Document how to schedule this with cron or a systemd timer (e.g. every 6/24 hours) — this is a documentation/config task, not new scheduling code; do not build a custom daemon/scheduler from scratch
- Each monitoring run creates a new `scan_id` linked to the same target, so Phase 2's `diff_scans()` can compare consecutive runs

6.2 Risk-Weighted Diff Scoring

- Create `modules/intel/risk_score.py` with `calculate_posture_score(findings_list)` that produces a single 0-100 score per scan, weighted by each finding's severity/CVSS (from Phase 1's NVD enrichment): start from 100 and subtract a weighted penalty per finding (e.g. critical CVSS 9-10 subtracts more than a low-severity header misconfiguration)
- Extend `diff_scans()` (Phase 2) to also return `risk_delta = new_score - previous_score`, so monitoring runs can flag 'posture got worse by X points' or 'improved by X points after remediation'
- Define a simple alert threshold (e.g. `risk_delta <= -5`, or any new CVSS ≥ 7.0 finding) that marks a monitoring run as 'ALERT' vs 'OK' in the database
- Each ALERT finding is passed through the Guided Remediation module (section 1.6) so the alert includes not just 'what changed' but 'why it matters and how to address it'

6.3 Dashboard

- Create a lightweight read-only web dashboard (Flask or FastAPI, single new file e.g. `dashboard.py`, run separately via `python3 dashboard.py`) that reads from the existing SQLite database — does not duplicate scanning logic
- Dashboard views: (1) posture score over time per target as a line chart, (2) findings count by severity as a bar chart, (3) a table of recent monitoring runs with ALERT/OK status and `risk_delta`, (4) drill-down into a single finding showing its remediation block
- Use a simple charting approach: Chart.js via CDN in a single HTML template (consistent with keeping this lightweight) — do not introduce a separate frontend build system/framework
- The dashboard is read-only and local-only (binds to 127.0.0.1); it is a reporting layer on top of existing data, not a new control plane
- This is the feature to highlight in academic demos: run a scan, show the score, fix a finding (or simulate fixing it), run again, show the score improve and the dashboard update — a visible before/after defensive story

3. Explicitly Out of Scope

- Do not copy code, tool lists, or architecture descriptions from HexStrike AI or any other third-party project verbatim. Original implementation only.
- Do not implement live exploit code generation or execution. Exploit reference mapping (Phase 3) is lookup/informational only.
- Do not bundle or embed the Odysseus AI workspace into this codebase in v4.0. MCP compatibility (Phase 4) makes external connection possible later without merging codebases.
- Do not remove or break any existing v3.0 menu options ([01]-[33]); all new functionality is additive.

4. How to Approach Implementation

- Work one phase at a time. After each phase, the project should still run via 'python3 main.py' without errors.
- When adding a new module, follow the existing pattern: a class or set of functions in `modules/<category>/<name>.py`, imported and added as a numbered option in `main.py`'s menu dictionary.
- Use the existing rich console output style (see `null_db/db.py` for examples of Table/Console usage) for any new terminal output.
- If a required file referenced in this brief was not attached, ask before guessing its contents — particularly `main.py`'s menu dispatch logic and `config/config.py`'s variable names.
- Test each new module's functions independently (e.g. via a small `__main__` block or simple script) before wiring into `main.py`, since there is no live system to run automated tests against in this conversation unless Claude Code with file execution is being used.

4.1 Self-Debugging Protocol (Important)

The user has limited technical troubleshooting experience and a limited budget for this session. To minimize back-and-forth, follow this protocol whenever an error occurs:

- If code produces an error during testing, do not stop and ask the user what to do. Read the full error message/traceback, check it against the attached repo files, and attempt a fix yourself.
- If the error is caused by a missing or mismatched function/variable name in the existing codebase, search the attached files for the correct name and adjust the new code to match — do not rename existing code unless necessary.
- If the error is caused by a missing Python package, add it to `requirements.txt` and note it in the final summary; do not assume the user can install packages interactively unless they confirm they can run pip commands.
- If the error is an API/auth/rate-limit issue (see Section on API & Cost Safety), apply the caching/fallback approach described there rather than treating it as a code bug.
- Only pause and ask the user a question if it is a genuine product decision (e.g. 'should Phase 3 also cover X') and cannot be inferred from this brief or the repo — not for technical/debugging decisions.
- At the end of each phase, provide a short plain-language summary: what was added, what file(s) changed, and a copy-pasteable command to test it (e.g. 'python3 main.py' then select option X).
- Acknowledge upfront: some trial-and-error during implementation is normal and expected, even with this brief. The goal of this protocol is to keep that trial-and-error self-contained, not to guarantee it won't happen.

5. Required Attachments

Attach this PDF together with the following files from the repository (github.com/rajatayyabb/NullShadow-v3-DL-RL). A zip of the whole repo is simplest; if attaching individually, these are the minimum required:

File	Why it's needed
<code>main.py</code>	Menu structure and dispatch — new options must follow this pattern
<code>null_db/db.py</code>	Current DB schema — Phase 1 extends this
<code>config/config.example.py</code>	Existing config variable names (LOCAL_AI_URL, DB_PATH, etc.)
<code>modules/ai/ai_engine.py</code>	Cloud AI interface pattern
<code>modules/ai/local_ai_engine.py</code>	Local AI interface — Phase 5 depends on this
<code>modules/pentesting/scanner.py</code>	Vulnerability scanner output format — Phase 1 normalizes this
<code>modules/osint/osint_tools.py</code>	CVE search tool — Phase 1 integrates NVD here
<code>modules/reporting/report_generator.py</code>	PDF report structure — Phase 2 diff output integrates here
<code>requirements.txt</code>	Current dependencies, to avoid version conflicts when adding new packages

Tip: in the new conversation, the simplest first message is something like: 'Here is the implementation brief for NullShadow v4.0 and a zip of the current repository. Please start with Phase 1.' Attach the PDF and the zip together.

Reminder: a new Claude conversation has no memory of prior conversations, even on the same account. This brief is designed to be self-contained so no prior context is required — everything needed should be in this document plus the attached repository files.